Community Library: Relational Schema

Software Engineering exercise: designing a database from a plain-English brief

The brief

A small community library lends books to members. It owns several physical copies of some titles, members borrow copies for a fixed period, and a member can join a queue for a title when every copy is out. The design needs to record books, copies, members, loans and reservations, and to cope honestly with a lost copy or a member who leaves.

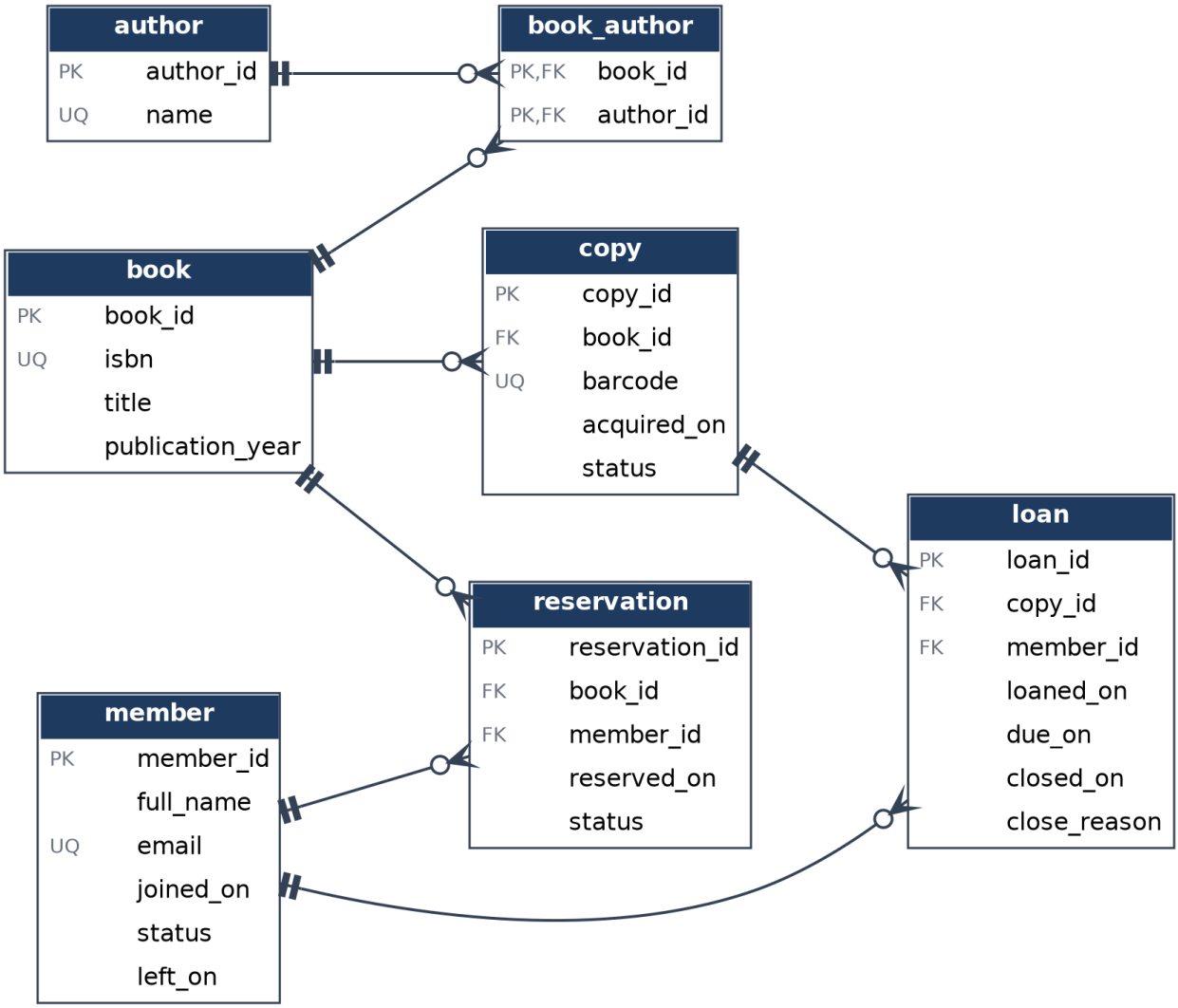
Assumptions I made

- A **book** is a title (the idea); a **copy** is one physical item with its own barcode. Loans attach to copies, not titles.
- A reservation is for a **title**, not a particular copy: whichever copy comes back first serves the queue.
- Whether a copy is “on loan” is **derived** from the loan table, never stored a second time, so it cannot drift out of sync.
- Loan length, fines and renewals are policy, so they live in the application and are out of scope here.

Design choices

- **Keys.** Every entity has a plain integer primary key. Natural identifiers (ISBN, barcode, email) are enforced with UNIQUE but are not used as primary keys, because an old book may have no ISBN and an email address can change. The junction table *book_author* uses a composite key.
- **No repeated data.** Authors live once in *author* and link to books through *book_author*, so a co-written book is two rows and a renamed author is one edit. Title details live on *book*, not on every copy.
- **Constraints do the policing.** CHECK constraints stop impossible dates and unknown statuses, foreign keys stop orphans, and two partial unique indexes enforce the rules that matter most: at most one **open** loan per copy, and at most one **live** reservation per member per title.
- **Nothing is hard-deleted.** Foreign keys use RESTRICT on the tables that hold history, so a member or copy with history cannot be erased by accident (tested below).

Entity diagram



Key: PK primary key, FK foreign key, UQ unique. A double bar means exactly one; a circle with a crow's foot means zero or many. Read *book* -|< *copy* as "a book has zero or many copies; each copy belongs to exactly one book".

CREATE TABLE statements

Written for SQLite and tested there. In PostgreSQL the same design works with DATE columns instead of ISO text, `GENERATED ALWAYS AS IDENTITY` keys, and the same partial indexes.

```
PRAGMA foreign_keys = ON;

CREATE TABLE author (
    author_id   INTEGER PRIMARY KEY,
    name        TEXT NOT NULL UNIQUE
);

CREATE TABLE book (
    book_id          INTEGER PRIMARY KEY,
    isbn             TEXT UNIQUE,                       -- optional: old or local books may have none
    title            TEXT NOT NULL,
    publication_year INTEGER CHECK (publication_year BETWEEN 1400 AND 2100)
);

CREATE TABLE book_author (
    book_id    INTEGER NOT NULL REFERENCES book(book_id)     ON DELETE CASCADE,
    author_id  INTEGER NOT NULL REFERENCES author(author_id) ON DELETE RESTRICT,
    PRIMARY KEY (book_id, author_id)
);

CREATE TABLE copy (
    copy_id      INTEGER PRIMARY KEY,
    book_id      INTEGER NOT NULL REFERENCES book(book_id) ON DELETE RESTRICT,
    barcode      TEXT NOT NULL UNIQUE,
    acquired_on  TEXT NOT NULL,                         -- ISO date, YYYY-MM-DD
    status       TEXT NOT NULL DEFAULT 'in_circulation'
                 CHECK (status IN ('in_circulation', 'lost', 'withdrawn'))
);

CREATE TABLE member (
    member_id  INTEGER PRIMARY KEY,
    full_name  TEXT NOT NULL,
    email      TEXT UNIQUE,                             -- NULL allowed (cleared when a member leaves)
    joined_on  TEXT NOT NULL,
    status     TEXT NOT NULL DEFAULT 'active'
               CHECK (status IN ('active', 'suspended', 'left')),
    left_on    TEXT,
    CHECK ((status = 'left') = (left_on IS NOT NULL))   -- left_on is set exactly when status is 'left'
);

CREATE TABLE loan (
    loan_id       INTEGER PRIMARY KEY,
    copy_id       INTEGER NOT NULL REFERENCES copy(copy_id)     ON DELETE RESTRICT,
    member_id     INTEGER NOT NULL REFERENCES member(member_id) ON DELETE RESTRICT,
    loaned_on     TEXT NOT NULL,
    due_on        TEXT NOT NULL,
    closed_on     TEXT,                                 -- NULL while the loan is open
    close_reason  TEXT CHECK (close_reason IN ('returned', 'lost')),
    CHECK (due_on >= loaned_on),
    CHECK (closed_on IS NULL OR closed_on >= loaned_on),
    CHECK ((closed_on IS NULL) = (close_reason IS NULL))
);
-- A physical copy can be out on at most one open loan at a time.
CREATE UNIQUE INDEX one_open_loan_per_copy ON loan(copy_id) WHERE closed_on IS NULL;
CREATE INDEX loan_by_member ON loan(member_id);

CREATE TABLE reservation (
    reservation_id  INTEGER PRIMARY KEY,
    book_id         INTEGER NOT NULL REFERENCES book(book_id)     ON DELETE RESTRICT,
    member_id       INTEGER NOT NULL REFERENCES member(member_id) ON DELETE RESTRICT,
    reserved_on     TEXT NOT NULL,
    status          TEXT NOT NULL DEFAULT 'waiting'
                    CHECK (status IN ('waiting', 'ready', 'fulfilled', 'cancelled', 'expired'))
);
-- A member can have only one live reservation per title.
CREATE UNIQUE INDEX one_live_reservation_per_member_title
    ON reservation(book_id, member_id) WHERE status IN ('waiting', 'ready');
```

Does the schema enforce what I claim?

I loaded the invented sample data (appendix) into SQLite, switched foreign keys on, and tried 12 statements that should either be refused or be accepted. All 12 behaved as intended.

Statement tried	Result from SQLite
Second open loan on a copy already out (copy 1)	REJECTED: UNIQUE constraint failed: loan.copy_id
Same member reserves same title twice while first is live	REJECTED: UNIQUE constraint failed: reservation.book_id, reservation.member_id
Due date before loan date	REJECTED: CHECK constraint failed: due_on >= loaned_on
Loan closed with no reason recorded	REJECTED: CHECK constraint failed: (closed_on IS NULL) = (close_reason IS NULL)
Copy with an invented status ('stolen')	REJECTED: CHECK constraint failed: status IN ('in_circulation', 'lost', 'withdrawn')
Duplicate barcode	REJECTED: UNIQUE constraint failed: copy.barcode
Member marked 'left' without a left_on date	REJECTED: CHECK constraint failed: (status = 'left') = (left_on IS NOT NULL)
Loan for a member that does not exist	REJECTED: FOREIGN KEY constraint failed
Hard-delete a member who has loan history (member 1)	REJECTED: FOREIGN KEY constraint failed
Hard-delete the departed member (member 5) who still has history	REJECTED: FOREIGN KEY constraint failed
Re-reserve a title after the earlier reservation was cancelled (member 4, book 4)	ACCEPTED
New loan for a copy whose earlier loan was returned (copy 6)	ACCEPTED

Three questions the design answers well

Results below are from the real run, with “today” fixed at 2026-10-26 so they can be reproduced.

1. Which loans are overdue today, who has them, and how late are they?

One join across loan, copy, book and member. The “open loan” test is a single column, *closed_on IS NULL*. The loan due today is correctly not overdue, and the lost-book loan is not listed because losing a copy closes its loan.

```
SELECT m.full_name, b.title, c.barcode, l.due_on,
       CAST(julianday('2026-10-26') - julianday(l.due_on) AS INTEGER) AS days_overdue
FROM loan l
JOIN copy c   ON c.copy_id   = l.copy_id
JOIN book b   ON b.book_id   = c.book_id
JOIN member m ON m.member_id = l.member_id
WHERE l.closed_on IS NULL AND l.due_on < '2026-10-26'
ORDER BY days_overdue DESC;
```

full_name	title	barcode	due_on	days_overdue
Priya Nair	The Lantern Keepers	LIB-0001	2026-10-19	7
Marco Ilyin	A Short Map of Rain	LIB-0005	2026-10-24	2

2. For each title, how many copies can be borrowed right now, and how many people are waiting?

Availability is computed from the copies and open loans, so it is always consistent. The Lantern Keepers has two copies, both out, with two people waiting. Salt and Signal shows 0 available: its only circulating copy is out, and the other copy is withdrawn.

```
SELECT b.title,
  (SELECT COUNT(*) FROM copy c WHERE c.book_id = b.book_id AND c.status = 'in_circulation') AS in_circulation,
  (SELECT COUNT(*) FROM copy c WHERE c.book_id = b.book_id AND c.status = 'in_circulation'
      AND NOT EXISTS (SELECT 1 FROM loan l WHERE l.copy_id = c.copy_id AND l.closed_on IS NULL)) AS available_now,
  (SELECT COUNT(*) FROM reservation r WHERE r.book_id = b.book_id AND r.status = 'waiting') AS waiting
FROM book b
ORDER BY available_now, waiting DESC, b.title;
```

title	in_circulation	available_now	waiting
The Lantern Keepers	2	0	2
Salt and Signal	1	0	1
A Short Map of Rain	2	1	0
Harbour of Small Hours	1	1	0
Notes on Quiet Engines	1	1	0
The Orchard Ledger	2	1	0

3. Which titles were borrowed most in 2026, by which authors, and by how many different members?

Authors come through the junction table, and a co-written book lists both. The loan made by the member who later left still counts, because history is kept.

```
SELECT b.title,
  (SELECT group_concat(a.name, ', ') FROM book_author ba JOIN author a ON a.author_id = ba.author_id
     WHERE ba.book_id = b.book_id) AS authors,
  COUNT(l.loan_id) AS loans_2026,
  COUNT(DISTINCT l.member_id) AS different_borrowers
FROM loan l
JOIN copy c ON c.copy_id = l.copy_id
JOIN book b ON b.book_id = c.book_id
WHERE l.loaned_on >= '2026-01-01' AND l.loaned_on < '2027-01-01'
GROUP BY b.book_id
ORDER BY loans_2026 DESC, b.title;
```

title	authors	loans_2026	different_borrowers
A Short Map of Rain	Amara Okonkwo-Reed, Dev Halvorsen	3	3
The Lantern Keepers	Ines Marlowe	3	3
Harbour of Small Hours	Kenji Aldous	2	2
The Orchard Ledger	Lucia Brandt	2	2
Notes on Quiet Engines	Dev Halvorsen	1	1
Salt and Signal	Tobias Quill	1	1

One question it answers badly

How many copies did the library have in circulation on 1 July 2026?

The *copy* table stores only a copy's **current** status, so the past cannot be reconstructed. The best query I can write is:

```
SELECT COUNT(*) AS copies_in_circulation_on_1_july_best_effort
FROM copy WHERE acquired_on <= '2026-07-01' AND status = 'in_circulation';
```

copies_in_circulation_on_1_july_best_effort
9

That answer of 9 is wrong. Copy 10 was lost on 1 August 2026, so it was in circulation on 1 July (loan 8 proves it was on loan in June), and copy 11 is now withdrawn but we do not know *when* it was withdrawn. The true answer is 10 or 11, and the schema cannot say which. A *copy_status_history* table (copy_id, status, effective_from) would fix it. I did not add it, because a small library rarely asks this and the extra table makes every status change a two-step write. I am noting it as a conscious trade-off.

Trade-offs and honest limits

When a book is lost

Marking a copy lost is two statements in one transaction: close its open loan with reason 'lost', then set the copy's status to 'lost'.

```
BEGIN;
UPDATE loan SET closed_on = '2026-10-26', close_reason = 'lost'
 WHERE copy_id = 5 AND closed_on IS NULL;
UPDATE copy SET status = 'lost' WHERE copy_id = 5;
COMMIT;
```

I ran this on copy 5 and the overdue list then correctly dropped that loan. **What it keeps:** the loan history and the fact the copy was lost. **What it does not do:** the schema has no fines or replacement cost, and it cannot stop someone setting a copy to 'lost' while leaving its loan open. That is the application's job, or a trigger I did not write. The date a copy was lost is only recoverable from the loan's *closed_on*, not from the copy.

When a member leaves

A member with history cannot be deleted: the foreign keys refuse it, and I tested both an active member and the departed one. Instead the row is anonymised in place:

```
UPDATE member
   SET full_name = 'Former member', email = NULL,
       status = 'left', left_on = '2026-10-26'
 WHERE member_id = 2;
```

Loans and reservations stay, so counts and popularity figures remain true, but the library can no longer say *who* borrowed what. That is the privacy gain and also the cost. The schema does not check that a leaving member has no open loans or live reservations, so staff or application code must settle those first.

Other limits

- Reservations name a title, not a copy, so the schema cannot record which returned copy is being held for someone, or until when.
- Queue position is derived by ordering on *reserved_on* among waiting reservations. It is correct, but it is a query, not a stored number.
- Computing availability needs a subquery each time. That is the price of never storing a value that could go stale.

Appendix: invented sample data

Every name, title, email and barcode is made up. Member 5 shows the ‘left’ state, copy 10 is lost and copy 11 is withdrawn.

```
-- All names, titles, emails and barcodes below are invented.
INSERT INTO author (author_id, name) VALUES
 (1,'Ines Marlowe'), (2,'Tobias Quill'), (3,'Amara Okonkwo-Reed'),
 (4,'Dev Halvorsen'), (5,'Lucia Brandt'), (6,'Kenji Aldous');

INSERT INTO book (book_id, isbn, title, publication_year) VALUES
 (1,'9780000000011','The Lantern Keepers',2018),
 (2,'9780000000028','Salt and Signal',2011),
 (3,'9780000000035','A Short Map of Rain',2020),
 (4,'9780000000042','Notes on Quiet Engines',2015),
 (5,NULL,'The Orchard Ledger',1994),
 (6,'9780000000066','Harbour of Small Hours',2022);

INSERT INTO book_author (book_id, author_id) VALUES
 (1,1),(2,2),(3,3),(3,4),(4,4),(5,5),(6,6);

INSERT INTO copy (copy_id, book_id, barcode, acquired_on, status) VALUES
 (1,1,'LIB-0001','2019-03-01','in_circulation'),
 (2,1,'LIB-0002','2019-03-01','in_circulation'),
 (3,2,'LIB-0003','2015-06-12','in_circulation'),
 (4,3,'LIB-0004','2021-01-20','in_circulation'),
 (5,3,'LIB-0005','2021-01-20','in_circulation'),
 (6,4,'LIB-0006','2016-09-05','in_circulation'),
 (7,5,'LIB-0007','2008-02-14','in_circulation'),
 (8,5,'LIB-0008','2012-11-30','in_circulation'),
 (9,6,'LIB-0009','2022-10-01','in_circulation'),
 (10,6,'LIB-0010','2022-10-01','lost'),
 (11,2,'LIB-0011','2015-06-12','withdrawn');

INSERT INTO member (member_id, full_name, email, joined_on, status, left_on) VALUES
 (1,'Priya Nair','priya.nair@example.org','2024-02-10','active',NULL),
 (2,'Samuel Ade','samuel.ade@example.org','2023-05-22','active',NULL),
 (3,'Hannah Voss','hannah.voss@example.org','2025-01-08','active',NULL),
 (4,'Marco Ilyin','marco.ilyin@example.org','2025-09-17','active',NULL),
 (5,'Former member',NULL,'2022-03-03','left','2026-08-30');

INSERT INTO loan (loan_id, copy_id, member_id, loaned_on, due_on, closed_on, close_reason) VALUES
 (1, 1,1,'2026-10-05','2026-10-19',NULL,NULL),
 (2, 2,2,'2026-10-20','2026-11-03',NULL,NULL),
 (3, 3,3,'2026-10-12','2026-10-26',NULL,NULL),
 (4, 4,1,'2026-09-01','2026-09-15','2026-09-14','returned'),
 (5, 5,4,'2026-10-10','2026-10-24',NULL,NULL),
 (6, 6,2,'2026-08-02','2026-08-16','2026-08-15','returned'),
 (7, 7,5,'2026-07-01','2026-07-15','2026-07-14','returned'),
 (8,10,3,'2026-06-10','2026-06-24','2026-08-01','lost'),
 (9, 9,4,'2026-09-20','2026-10-04','2026-10-03','returned'),
 (10,4,2,'2026-10-01','2026-10-15','2026-10-16','returned'),
 (11,1,3,'2026-09-10','2026-09-24','2026-09-20','returned'),
 (12,8,1,'2026-10-22','2026-11-05',NULL,NULL);

INSERT INTO reservation (reservation_id, book_id, member_id, reserved_on, status) VALUES
 (1,1,3,'2026-10-21','waiting'),
 (2,1,4,'2026-10-23','waiting'),
 (3,3,2,'2026-09-28','fulfilled'),
 (4,4,4,'2026-10-01','cancelled'),
 (5,2,1,'2026-10-24','waiting');
```